

Technical Configuration Document

Microsoft 365 Email Integration via OAuth 2.0 & Microsoft Graph API

(Date 19 - March - 2026)

Contents

Overview	3
System Components.....	3
OAuth 2.0 Configuration	3
Azure App Registration	3
API Permissions	3
Environment Variables (.env)	3
Access Token Management	4
Token Endpoint	4
Grant Type.....	4
Scope	4
Token Storage Strategy.....	4
Token Reuse Logic	4
Microsoft Graph API Implementation.....	4
Endpoint Used	4
Request Headers	5
Response Handling	5
Packages / Dependencies	5
Error Handling	5
Security Considerations	5
Architecture Flow	5
Network Requirements	6
Required Endpoints (Allowlist)	6
Summary.....	6

Overview

This document describes the implementation of email sending functionality using **OAuth 2.0 Client Credentials Flow** and **Microsoft Graph API** within the Laravel-based ticketing system.

The integration replaces traditional SMTP authentication with a secure token-based approach.

System Components

- Backend Framework: Laravel (PHP)
- Database: MySQL
- Authentication: OAuth 2.0 (Client Credentials Flow)
- API Used: Microsoft Graph API
- Email Source Account: customerrelations@siyapatha.lk

OAuth 2.0 Configuration

Azure App Registration

Configured in Azure Active Directory (Microsoft Entra ID) with:

- Application (Client) ID
- Directory (Tenant) ID
- Client Secret

API Permissions

The following **Application Permissions** are required:

- Mail.Send

Environment Variables (.env)

MICROSOFT_TENANT_ID=your-tenant-id

MICROSOFT_CLIENT_ID=your-client-id

MICROSOFT_CLIENT_SECRET=your-client-secret

Access Token Management

Token Endpoint

https://login.microsoftonline.com/{tenant_id}/oauth2/v2.0/token

Grant Type

client_credentials

Scope

<https://graph.microsoft.com/.default>

Token Storage Strategy

- Tokens are stored in DB table: access_token_365
- Fields:
 - access_token
 - expires_in
 - ext_expires_in
 - status
 - update_datetime

Token Reuse Logic

- Token reused until expiry
- 60-second buffer applied before expiration
- Expired tokens marked as inactive
- New token automatically generated

Microsoft Graph API Implementation

Endpoint Used

POST <https://graph.microsoft.com/v1.0/users/{email}/sendMail>

Example:

<https://graph.microsoft.com/v1.0/users/customerrelations@siyapatha.lk/sendMail>

Request Headers

Authorization: Bearer {access_token}

Content-Type: application/json

Response Handling

- **202 Accepted** → Email successfully sent
- Other responses → Logged as failure with HTTP code and response body

Packages / Dependencies

No external packages installed.

Uses native cURL and Laravel components.

Error Handling

- cURL errors captured
- HTTP status validation
- Token failure handling
- JSON response logging

Security Considerations

- Secrets stored in .env (not hardcoded)
- HTTPS enforced for API calls
- Token reuse minimizes exposure

Architecture Flow

User Request → Laravel App → DB Token Check → Microsoft OAuth → Graph API → Recipient

Network Requirements

Required Endpoints (Allowlist)

- <https://login.microsoftonline.com>
- <https://graph.microsoft.com>

Summary

- Secure email sending implemented via OAuth 2.0
- No dependency on SMTP credentials
- Token-based authentication with automatic refresh
- Fully integrated with Microsoft 365 ecosystem